



Practica 2

Práctica 2 - HTTP.pdf

1. ¿Cuál es la función de la capa de aplicación?

- Define formato de los mensajes → Algunos protocolos usan ASCII como HTTP y otros por ejemplo usan ASN
- Define semántica de los mensajes
- Define como debe ser el intercambio de mensajes, es decir, que mensajes se intercambian

2. Si dos procesos deben comunicarse:

a. ¿Cómo podrían hacerlo si están en diferentes máquinas?

- Dos procesos que se encuentran en diferentes máquinas se comunican a través del envío de mensajes mediante una red de computadoras. Uno actúa como emisor, creando y enviando mensajes a la red, y el otro actúa como receptor, recibiendo mensajes y posiblemente respondiendo a los mismos

b. Y si están en la misma máquina, ¿qué alternativas existen?

- Si están en la misma máquina se pueden comunicar a través de interprocesos, aplicando reglas gobernadas por el SO del sistema terminal

3. Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la "vida cotidiana" y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

- El modelo cliente/servidor cuenta con un host activo, denominado servidor, el cual cuenta con una IP. Este último brinda servicios a las solicitudes de muchos otros hosts, los cuales son los clientes y lo hace

mediante la recepción y servicio de solicitudes de estos clientes. Debido a que el servidor cuenta con una IP fija y conocida y siempre está activo, un cliente siempre puede contactar con él enviando un paquete a esa IP

-Normalmente un solo servidor no puede responder a todas las solicitudes de sus clientes, por lo que en las arquitecturas de este tipo de modelo cliente/servidor se suele utilizar un centro de datos el cual cuenta con una gran cantidad de hosts

-Este modelo se ve reflejado en la vida cotidiana por ejemplo cuando vas a comprar a un kiosco o una tienda, ya que el cliente pediría ciertos productos y el servidor (empleado) se los brindaría en caso de poder dárselos. Por otro lado, un ejemplo de un sistema informático que siga este modelo sería una web app, donde el cliente sería el navegador y el servidor sería el servidor web

-P2P es el otro modelo de comunicación que conozco, donde los clientes y los servidores pueden actuar como ambos para compartir recursos directamente entre sí

4. **Describa la funcionalidad de la entidad genérica "Agente de usuario" o "User agent".**

-User Agent → Aplicación o software que actúa en nombre del usuario en un sistema o red. Es un componente que representa al usuario en sus interacciones con servicios en línea, sitios web u otros sistemas. Se refiere principalmente al navegador que un usuario emplea para entrar a los sitios web. El User Agent comunica al servidor detalles de ese navegador, como la versión, y detalles sobre el sistema operativo/dispositivo. Con esta información los sitios adaptan su contenido a las capacidades del navegador

-En una request HTTP del navegador, el User Agent es un encabezado que informa sobre el navegador y sistema operativo

5. **¿Qué son y en qué se diferencian HTML y HTTP?**

-HTTP → Protocolo de transferencia de hipertexto, es el protocolo de la capa de la aplicación web y se implementa entre dos programas: programa cliente y un programa servidor, los cuales se ejecutan en terminales diferentes y se comunican entre sí intercambiando mensajes HTTP. Estos mensajes tienen una estructura definida y una forma en como el cliente y servidor los intercambian

-HTML → Lenguaje de marcado utilizado para crear la estructura y contenido de las paginas web, siendo este el mas utilizado por la mayoría de paginas web en un archivo base y varios objetos referenciados. Si una pagina tiene texto html y 5 imagenes, en total va a tener 5 objetos, el archivo base HTML y las 5 imagenes. El archivo HTML hace referencia a los otros objetos contenidos en la pagina mediante la URL de los objetos, la cual tiene dos compoonentes: nombre del host del servidor que alberga el archivo y nombre de la ruta al objeto

6. HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas. (Ayuda: apartado "Formato de mensaje HTTP", Kurose).

a. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información? ¿Para qué sirven las cabeceras?

-La informacion que indica si un mensaje HTTP es de requerimiento o de respuesta se encuentra en la primera linea, la cual esta compuesta por:

-Requerimiento: Metodo + recurso solicitado (URL) + version de HTTP → `GET /index.html HTTP/1.1`

-Respuesta la primea linea estaria compuesta por Version HTTP + Codigo de estado + frase → `HTTP/1.1 200 OK`

-Las cabeceras o headers sirven para transportar informacion adicional sobre el requerimiento o la respuesta, indicando datos necesarios para que el cliente y el servidor puedan interpretar y procesar correctamente los mensajes. Contiene informacion tales como el Host, User Agent, tipo de contenido, cookies, etc

b. ¿Cuál es su formato? (Ayuda: <https://developer.mozilla.org/es/docs/Web/HTTP/Headers>)

-Un mensaje HTTP tiene la siguiente estructura general:

```
Linea de requerimiento
Cabecera: valor
Cabecera: valor
...

Linea en blanco
```

Cuerpo del mensaje (opcional)

-La primera línea tiene Metodo + URL + Version HTTP y luego aparecen las cabeceras. Luego de las cabeceras hay una línea en blanco que indica el final de las mismas y puede existir un body el cual es opcional. En un POST podría aparecer para enviar información. Ejemplo de la teoría:

```
POST /index.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
Content-Type: text/plain
Content-Length: 10

1234567890

HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:37:22 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
```

-La primera línea de respuesta acá tiene Version HTTP + Código de Estado + Frase. El mensaje de Requerimiento y de Respuesta se separa por líneas en blanco

- c. **Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como www.misitio.com para obtener el recurso `/index.html`. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento.**

-Se debería enviar la siguiente información en el requerimiento:

```
GET /index.html HTTP/1.1

Host: www.misitio.com
User-Agent: curl/7.74.0
```

7. **Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s).**

HTTP cuenta con un comando HEAD que se usa para obtener nada mas que el encabezado de un documento. Cuando se usa en un archivo FTP o FILE, curl muestra el size del archivo y la ultima hora de modificacion

- `-X, --request <command>` → Especifica metodo de solicitud personalizado que se utilizara al comunicarse con el servidor HTTP. Por default, CURL usa GET

```

redes@debian:~$ curl -X GET http://www.google.com/
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="es-419"><head><meta content="text/html; charset=UTF-8" http-equiv="Content-Type"><meta content="images/branding/google/4x/google_standard_color_128dp.png" itemprop="image"><title>Google</title><script nonce="yUkFYzRWgZnLKMHDK41Iow">(function){var g={key:'WV6vAOZCOZL5U0PJpLGAU0E',xKXPt:'0,18168,4316795,344796,219810,5312785,12,193,539,55,486,5991185,9,29,30819265,25228681,152392,65154,138189,8171,37791,107648,41099,26230,25591,46339,20677,11884,16179,7166,35614,4,1514,3355,12011,16,17398,5089,927,18,777,36623,5337,2892,1,1,6,8522,19726,5,964,2,147,34657,2,3,6332,14329,3,4980,10,13528,6293,16983,6445,11427,1,930,12246,10,1717,3933,21617371,4,2960,3,10055,3,17894,24,50,3,3780,2,6508412,6,8302,8370,3,452,2528,1249,3,1567,2235,2,3312,570,3,497,1242,632,134,285,1530,3,377,880,227,75554,2051075,1464697,915035,12406993,2,2195242,1,105686,197111,2,3269692,5,73958,3589,47943,3,196172,388,121544,656969,1073,2,1721,6776,7,7,16845,10,1237,1109,4895,5930,377,4011,5,18,15698,11451,5,7854,4,4988,5,4036,6328,6251,1685,3396,1864,1734,7889,955,2139,4,4585,911,4329,1505,1531,7490,28,3,515,6781,7567,2160,4,10526,280,5733,9542,4,1483,2475,5853,5434,3238,4,26,278,57,5,695,5,58,4,996,2319,2624,2048,4,4,421,4,3196,5,266,1564,1,2603,1,689,221,5,1136,4445,5,1486,129,4,4829,2762,7,559,6,2349,1,2,1,1,8,1,3744,5,642,414,4,1252,12,252,3520,7,4,131,4,480,4,2855,4,2002,101,2,453,5,202,4,5688,2308,3968,4,933,4,69,5,5253,360,601,1637,2879,5,1253,5,653,4,2993,804,276,4,191,3493,5,3090,1,563,5,1293,5,31,5947,4,9619,825,2,2694,1071,5,123,2627,4,1566,2061,9,74,781,931,2,281,4,2047,4,1478,4,1634,1168,4,177,4,2698,4,1410,3942,5,2076,5,2587,7,11,2123,3258,1289,4,571,236,591,5,578,3,2,2,2,381,3,2,2,1,2,3,4,4208,10,1852,3,1728,4,2700,1,1,1,2,1,005,3099,4,153,6,2684,2268,3,2,2,2,221,4,761,28,20,189,3,2,4,43,83,4,1936,478,333,2083,950,1162,4,2737,82,3010,402,553,4,512,4,1126,2335,199,59,3,2,2,2,929,460,82,3,3,1188,1586,333,5,1117,22,4066,5,240,422,1,2137,1561,733,3786,6,119,2,3,2,2,2,259,190,5,884,333,779,1507,36,186,70,8,4,27,4,303,2758,4,51,233,58,4,5,1083,499,1774,5,615,2521,134,4,13,642,592,4,1025,49,4,608,477,4,914,4,2,24,39,4,2,52,4,225,270,1751,4,1237,4,712,3,2,2,2,3123,4497,1462,114,1400,4,753,2,1437,92,81,270,162,1226,752,1133,463,1,367,4,2,145,4,224,174,1237,994,580,24,188,55,4,37,184,456,3,2,2,2,108,60,134,47,4,558,697,155,5,7,4,472,483,106,3,2,2,2,22,77,786,55,4,1340,8,829,4,1349,231,23,2139,4,13,312,5,663,4,2,253,4,10,400,49,4,19,1108,4,395,5,153,340,193,1219,2,6,71,431,3,121,4,2,796,5,2,1095,426,127,570,304,9,2091,4,895,20,3,2,2,2,46,2,56,4,150,1550,659,143,4,2,1120,195,162,11,12,213,4,74,10,463,4,976,249,1832,4,555,4,120,56,255,3,2,2,2,384,638,201,295,394,500,120,1,1,2,19,1713,4,749,75,6,91,2243,88,665,4,1604,1436,206,1009,1872,34,2,9,12,209,2948,3196,829,4464,'kBL':ArS-,'KLOP':89978449;}(function){var a;((a=window.google)=null?0:a.stvs)>?google.KEY=g.KEY:(window.google=g;)).call(this);})();(function){google.sn='webhp';google.kHL='es-419';google.usb=true;})();(function){if var={this|self;var k=#/$/;function l(a,c){if d=a.a.search(k);a;{var b=0;for(var f=c.length;(b=a.indexOf(c,b))>0&&b<0;var d=a.a.charCodeAt(b-1)|d==38|f==63|if d=a.a.charCodeAt(b+f)|d|d==61|d==38|d==35)break a;b+=f+1;b=1;if(b<0)return null;f=a.a.indexOf("&#",b);if(f<0|f>e)f=b+c.length+1;return decodeURIComponent(a.slice(b,f));f==17f:0).rep

```

- `-s, --silent` → Modo silencioso de CURL, no muestra barra de progreso ni mensajes de error. Silencio Curl pero seguira generando los datos que solicita, incluso al terminal/salida estandar a menos que se redirija. Util para salidas mas limpias, donde solo deseeas ver la respuesta del servidor

8. Ejecute el comando curl sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:

- a. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (>) del comando curl a un archivo con extensión html y abrirlo con un navegador.

```
redes@debian:~$ curl www.redes.unlp.edu.ar
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>...Redes y Comunicaciones...Facultad de Inform&aacute;tica...UNLP...</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="">
<meta name="author" content="">

<!-- Le styles -->
<link href="/bootstrap/css/bootstrap.css" rel="stylesheet">
<link href="/css/style.css" rel="stylesheet">
<link href="/bootstrap/css/bootstrap-responsive.css" rel="stylesheet">

<!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
<!--[if lt IE 9]>
<script src="/bootstrap/js/html5shiv.js"></script>
<![endif]-->
</head>

<body>

<div id="wrap">
<div class="navbar navbar-inverse navbar-fixed-top">
<div class="navbar-inner">
<div class="container">
<a class="brand" href="/index.html"><i class="icon-home icon-white"></i></a>
<a class="brand" href="https://catedras.info.unlp.edu.ar" target="_blank">Redes y Comunicaciones</a>
<a class="brand" href="http://www.info.unlp.edu.ar" target="_blank">Facultad de Inform&aacute;tica</a>
<a class="brand" href="http://www.unlp.edu.ar" target="_blank">UNLP</a>
</div>
</div>
</div>

<div class="container">
```

-Al no poner ningun parametro CURL usa GET y esto nos devuelve un HTML del servidor www.redes.unlp.edu.ar, realizando solo 1 requerimiento. Luego de redireccionarlo a un archivo html y abrirlo muestra esto:



b. **¿Cómo funcionan los atributos href de los tags link e img en html?**

-El atributo href en la etiqueta <link> se usa para enlazar recursos externos y el atributo src se usa para especificar la ubicación de la imagen que se mostrara en la web. En ambos casos el navegador realiza requests HTTP/HTTPS para descargar los recursos externos de estos atributos. Dependiendo del tipo de recurso, realiza acciones adicionales como agregarle estilos en el caso de las hojas o mostrar imagenes en el caso de las etiquetas

c. **Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?**

-No alcanza, se necesita pedir cada recurso (archivo, imagen, JS, CSS) por separado al servidor

d. **¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos CSS, dos Javascript y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando curl ejecutado previamente.**

-Se necesitaran en total:

- 1 Requerimiento del archivo base HTML
- 2 Requerimientos para CSS
- 2 Requerimientos para JS
- 3 Requerimientos para las imagenes

-En un navegador, al recibir el HTML, este analiza las etiquetas link, img, script, detecta los recursos externos y realiza automáticamente los requerimientos necesarios para obtenerlos y mostrar la página completa. Con curl, el requerimiento realizado previamente obtiene solamente el recurso solicitado (HTML). No interpreta el HTML para solicitar automáticamente los CSS, JavaScript e imágenes, se tienen que realizar los requerimientos de esos recursos por separado

9. Ejecute a continuación los siguientes comandos:

curl -v -s www.redes.unlp.edu.ar > /dev/null

curl -I -v -s www.redes.unlp.edu.ar

a. ¿Qué diferencias nota entre cada uno?

```
redes@debian:~$ curl -v -s www.redes.unlp.edu.ar > /dev/null
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Wed, 02 Sep 2026 18:03:46 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
{ [4898 bytes data]
* Connection #0 to host www.redes.unlp.edu.ar left intact
```

```
redes@debian:~$ curl -I -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> HEAD / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
HTTP/1.1 200 OK
< Date: Wed, 02 Sep 2026 18:04:37 GMT
Date: Wed, 02 Sep 2026 18:04:37 GMT
< Server: Apache/2.4.56 (Unix)
Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
Accept-Ranges: bytes
< Content-Length: 4898
Content-Length: 4898
< Content-Type: text/html
Content-Type: text/html
```

-La diferencia principal es que el primer comando realiza una petición GET, para enviar `GET /HTTP1.1` y lo redirige a /dev/null, por lo que el contenido no aparece en pantalla. En cambio con el segundo comando realiza una petición HEAD, para enviar `HEAD / HTTP1.1` y recibe solo los headers pero no el HTML

b. **¿Qué ocurre si en el primer comando se quita la redirección a/dev/null? ¿Por qué no es necesaria en el segundo comando?**

-Si se quita la redirección obtengo el HTML completo. Como mencione antes esto antes no se mostraba en pantalla ya que era redireccionado a ese archivo. Esta redirección no es necesaria en el 2do comando ya que con el parametro `-I` solo se hace una petición HEAD y no va a pedir el HTML completo

```
redes@debian:~$ curl -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Wed, 02 Sep 2026 18:11:51 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>...Redes y Comunicaciones...Facultad de Inform&aacute;tica...UNLP...</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">

    <!-- Le styles -->
    <link href="/bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="/css/style.css" rel="stylesheet">
    <link href="/bootstrap/css/bootstrap-responsive.css" rel="stylesheet">

    <!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
    <!--[if lt IE 9]>
      <script src="/bootstrap/js/html5shiv.js"></script>
```

c. **¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?**

-En el primer comando:

-Requerimiento:

```
Host: www.redes.unlp.edu.ar
User-Agent: curl/7.74.0
Accept: */*
```

-Respuesta:

```
Date: We, 02 Sep 2026 18:03:46 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:26 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 4898
Content-Type: text/html
```

10. ¿Qué indica la cabecera Date?

-La cabecera Date indica fecha y hora en la que se generó la solicitud o respuesta HTTP

11. En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?

-En HTTP/1.0 → El cliente se da cuenta que recibió todo el objeto solicitado de manera completa cuando el servidor cierra la conexión luego de enviar la respuesta

-En HTTP/1.1 → Se introducen mejoras para manejar esto. El encabezado "Content-Length" indica la longitud en bytes del objeto de la respuesta, dándole una idea de cuántos datos debe esperar al cliente. "Transfer-Encoding" con valor "chunked" divide la respuesta en partes y el cliente detecta el final de la respuesta cuando recibe una parte con un tamaño de 0.

12. Investigue los distintos tipos de códigos de retorno de un servidor web y su significado. Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).

●	1XX	Códigos informativos	El servidor ha recibido la petición y procederá con ella.
●	2XX	Códigos de éxito	El servidor ha recibido, entendido y procesado la solicitud correctamente.
●	3XX	Códigos de redirección	El servidor ha recibido la solicitud, pero hay una redirección a alguna otra parte (o, en raras ocasiones, alguna acción adicional que debe completarse).
●	4XX	Códigos de error de cliente	El servidor no puede encontrar (o alcanzar) la página o la web. Se trata de un error del lado de la web.
●	5XX	Códigos de error de servidor	El cliente ha realizado una solicitud válida, pero el servidor ha fallado al completarla.

```

HTTP/<version> 200 OK
HTTP/<version> 301 Moved Permanently
HTTP/<version> 400 Bad Request
HTTP/<version> 403 Access Forbidden
HTTP/<version> 404 Not Found
HTTP/<version> 405 Method Not Allowed
HTTP/<version> 500 Internal Server Error (CGI Error)
HTTP/<version> 501 Method Not Implemented

```

13. Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:

```

redes@debian:~$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 18:27:00 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 4898
Content-Type: text/html

```

1. ¿Qué información brinda la primera línea de la respuesta?

-En la primera línea se indica HTTP/1.1 200 OK, indicando versión HTTP (1.1), Código de Estado (200) y frase (OK)

2. ¿Cuántos encabezados muestra la respuesta?

-Muestra 7 → Date, Server, Last-Modified, ETag, Accept-Ranges, Content-Length/Type

3. ¿Qué servidor web está sirviendo la página?

-El servidor web que utiliza la página es Apache/2.4.45 (Unix)

4. ¿El acceso a la página solicitada fue exitoso o no?

-Debido a que el código de estado fue 200 y la frase OK esto indica que el acceso a la página fue exitoso/satisfactorio

5. ¿Cuándo fue la última vez que se modificó la página?

-El 19 de Marzo de 2023 a las 19:04:46 GMT, lo indica Last-Modified

6. Solicite la página nuevamente con curl usando GET, pero esta vez indique que quiere obtenerla sólo si la misma fue modificada en una fecha posterior a la que efectivamente fue modificada. ¿Cómo lo hace? ¿Qué resultado obtuvo? ¿Puede explicar para qué sirve?

-Para indicar que me de un requerimiento en tal fecha específica se debe usar el header "If-Modified-Since: <fecha> ". Podría servir para comparar una versión anterior con una nueva luego de ser modificada, aunque en este caso no lo fue ya que el 304 nos indica "Not Modified". El primer comando no recibe nada ya que le pide un GET si fue modificada y como no lo fue no le devuelve nada, por eso use -I para pedirle información sobre el recurso y que muestre algo

```
redes@debian:~$ curl -H "If-Modified-Since: Sun, 19 Mar 2023 19:04:46 GMT" www.redes.unlp.edu.ar
redes@debian:~$ curl -I -H "If-Modified-Since: Sun, 19 Mar 2023 19:04:46 GMT" www.redes.unlp.edu.ar
HTTP/1.1 304 Not Modified
Date: Wed, 02 Sep 2026 18:35:04 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
redes@debian:~$
```

14. Utilizando curl, acceda al sitio

www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados.

-En el header Authorization Basic las credenciales se constituyen como user:password y las codifican en base64, por lo que hay que transcribir ese texto y con el pipeline con el comando base64 lo transcribe. Como pide buscar la URL oculta creo que se podría hacer buscando directamente en el HTML de la URL pero el comando -v lo mostraría ↓↓↓

```
redes@debian:~$ curl http://www.redes.unlp.edu.ar/restringido/index.php
<h1>Acceso restringido</h1>

<p>Para acceder al contenido es necesario autenticarse. Para obtener los datos de acceso seguir las instrucciones detalladas en www.redes.unlp.edu.ar/obtener-usuario.php</p>
redes@debian:~$ curl www.redes.unlp.edu.ar/obtener-usuario.php
<p>Para obtener el usuario y la contraseña haga un requerimiento a esta página seteando el encabezado 'Usuario-Redes' con el valor 'obtener'</p>
redes@debian:~$ curl -H "Usuario-Redes:obtener" www.redes.unlp.edu.ar/obtener-usuario.php
<p>Bien hecho! Los datos para ingresar son:

    Usuario: redes
    Contraseña: RYC

    Ahora vuelva a acceder a la página inicial con los datos anteriores.

    PISTA: Investigue el uso del encabezado Authorization para el método Basic. El comando base64 puede ser de ayuda!</p>
redes@debian:~$ echo -n "redes:RYC" | base64
cmVkZXN6M6U1LD
redes@debian:~$ curl -H "Authorization: Basic cmVkZXN6M6U1LD" http://www.redes.unlp.edu.ar/restringido/index.php
<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACIÓN: la URL de la siguiente página está contenida en esta misma respuesta.</p>
redes@debian:~$
```

-Ahora usamos el parametro `-v` para mostrar informacion detallada de la comunicacion cliente/servidor y muestra en Location la URL escondida

```
redes@debian:~$ curl -v -H "Authorization: Basic cmVkZXN6M6U1LD" http://www.redes.unlp.edu.ar/restringido/index.php
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/index.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXN6M6U1LD
* Mark bundle as not supporting multiuse
< HTTP/1.1 302 Found
< Date: Wed, 02 Sep 2026 18:57:28 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Location: http://www.redes.unlp.edu.ar/restringido/the-end.php
< Content-Length: 230
< Content-Type: text/html; charset=UTF-8
<
<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACIÓN: la URL de la siguiente página está contenida en esta misma respuesta.</p>
```

```
redes@debian:~$ curl -v -H "Authorization: Basic cmVkZXN6M6U1LD" http://www.redes.unlp.edu.ar/restringido/the-end
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/the-end.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXN6M6U1LD
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Wed, 02 Sep 2026 18:58:44 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Content-Length: 159
< Content-Type: text/html; charset=UTF-8
<
¡Felicitaciones, llegaste al final del ejercicio!

Fecha: 2026-09-02 18:58:44
* Connection #0 to host www.redes.unlp.edu.ar left intact
Verificación: d1a46a1ea71ee0dda487e9995df42002979614b13cb1b7fe24c9f1487e986829redes@debian:~$
```

15. Utilizando la VM, realice las siguientes pruebas:

- a. Ejecute el comando 'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt' y copie la salida completa (incluyendo los dos saltos de línea del final).
- b. Desde la consola ejecute el comando telnet www.redes.unlp.edu.ar 80 y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de hacerlo?

```
redes@debian:~$ telnet www.redes.unlp.edu.ar 80
Trying 172.28.0.50...
Connected to www.redes.unlp.edu.ar.
Escape character is '^]'.
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*

HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 19:32:50 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "760-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 1888
Connection: close
Content-Type: text/html

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Protocolo HTTP: versiones</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">

    <!-- Le styles -->
    <link href="../../bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="../../css/style.css" rel="stylesheet">
    <link href="../../bootstrap/css/bootstrap-responsive.css" rel="stylesheet">

    <!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
    <!--[if lt IE 9]>
      <script src="../../bootstrap/js/html5shiv.js"></script>
```

```

</head>
<body>

  <div id="wrap">
    <div class="navbar navbar-inverse navbar-fixed-top">
      <div class="navbar-inner">
        <div class="container">
          <a class="brand" href="../../index.html"><i class="icon-home icon-white"></i></a>
          <a class="brand" href="https://catedras.info.unlp.edu.ar" target="_blank">Redes y Comunicaciones</a>
          <a class="brand" href="http://www.info.unlp.edu.ar" target="_blank">Facultad de Inform&acut;tica</a>
          <a class="brand" href="http://www.unlp.edu.ar" target="_blank">UNLP</a>
        </div>
      </div>
    </div>

    <div class="container">
      <h1>Ejemplo del protocolo HTTP 1.1</h1>
      <p>
        Esta p&acut;gina se visualiza utilizando HTTP 1.1. Utilizando el capturador de paquetes analice cuantos flujos utiliza
        l navegador para visualizar la p&acut;gina con sus im&acut;genes en contraposici&acut;n con el protocolo HTTP/1.0.
      </p>
      <p>
        <h2>Imagen de ejemplo</h2>
        
      </p>
    </div>

    <div id="footer">
      <div class="container">
        <p class="muted credit">Redes y Comunicaciones</p>
      </div>
    </div>
  </body>
</html>
Connection closed by foreign host.

```

- c. Repita el proceso anterior, pero copiando la salida del recurso `/extras/prueba-http-1-1.txt`. Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando `telnet` nuevamente.

-Ocurre lo mismo pero sin cierre de conexion luego del envio de respuesta

16. En base a lo obtenido en el ejercicio anterior, responda:

- a. ¿Qué está haciendo al ejecutar el comando `telnet`?

-El comando `telnet` intenta establecer una conexion con el servidor "www.redes.unlp.edu.ar" en el puerto 80. Es una herramienta para conectarse a servidores al igual que SSH (el cual permite establecer una conexion con otro servidor y ejecutar comandos) pero lo hace en texto claro, de manera insegura, mientras que SSH lo hace de manera cifrada

- b. ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?

-Se utilizo el metodo `GET` y se solicito el recurso `/http/HTTP-1.1/`

```

GET /http/HTTP-1.1/HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*

```

- c. ¿Qué diferencias notó entre los dos casos? ¿Puede explicar por qué?

-La diferencia que hay entre los dos casos es que la conexión no se cierra una vez que el servidor envía la respuesta en el inciso 15-c, cuando usas el txt del http 1.1. Esto ocurre debido a que, a diferencia de HTTP 0 (donde cada solicitud y respuesta abre y cierra la conexión), en HTTP 1.1 se puede usar una sola conexión para realizar varias operaciones

d. **¿Cuál de los dos casos le parece más eficiente? Piense en el ejercicio donde analizó la cantidad de requerimientos necesarios para obtener una página con estilos, javascripts e imágenes. El caso elegido, ¿puede traer asociado algún problema?**

-El caso mas eficiente por supuesto seria el de HTTP 1.1, ya que reducimos la carga y descarga de conexión para cada solicitud, disminuyendo la sobrecarga de la misma y conduciendo a una carga mas rapida de paginas web y mejor uso de los recursos de la red

-El problema asociado a la conexión persistente ocurre cuando se solicita un recurso muy grande, ya que se mantiene la conexión durante un periodo muy prolongado y podría bloquear/retrasar la obtención de los demás recursos. Esto se conoce como HOL (Head-of-line blocking). Los navegadores modernos utilizan técnicas como pipelining/multiplexación para solucionar este tipo de inconvenientes, donde se permite la transferencia de múltiples recursos a la vez

17. En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET. Para ello, será necesario utilizar la VM y la herramienta Wireshark. Antes de iniciar considere:

- **Capture los paquetes utilizando la interfaz con IP 172.28.0.1. (Menú "Capture → Options". Luego seleccione la interfaz correspondiente y presione Start).**
- **Para que el analizador de red sólo nos muestre los mensajes del protocolo http introduciremos la cadena 'http' (sin las comillas) en la ventana de especificación de filtros de visualización (display-filter). Si no hiciéramos esto veríamos todo el tráfico que es capaz de capturar nuestra placa de red. De los paquetes que son capturados, aquel que esté seleccionado será mostrado en forma detallada en la sección que está justo debajo. Como sólo estamos interesados en http ocultaremos toda la información que no es relevante para esta práctica (Información de trama, Ethernet, IP y TCP). Desplegar la**

información correspondiente al protocolo HTTP bajo la leyenda "Hypertext Transfer Protocol".

- Para borrar la cache del navegador, deberá ir al menú "Herramientas→Borrar historial reciente". Alternativamente puede utilizar Ctrl+F5 en el navegador para forzar la petición HTTP evitando el uso de caché del navegador.
 - En caso de querer ver de forma simplificada el contenido de una comunicación http, utilice el botón derecho sobre un paquete HTTP perteneciente al flujo capturado y seleccione la opción Follow TCP Stream.
1. Abra un navegador e ingrese a la URL: www.redes.unlp.edu.ar e ingrese al link en la sección "Capa de Aplicación" llamado "Métodos HTTP". En la página mostrada se visualizan dos nuevos links llamados: Método GET y Método POST. Ambos muestran un formulario como el siguiente:



Nombre

Apellido

Email

Sexo Masculino: ☒ Femenino: ☐

Contraseña

Recibir confirmaciones por email ☐

2. Analice el código HTML

-GET:

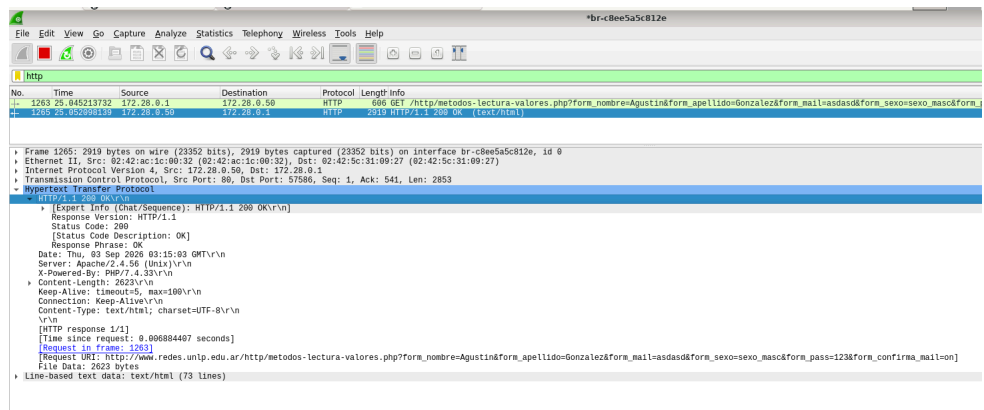
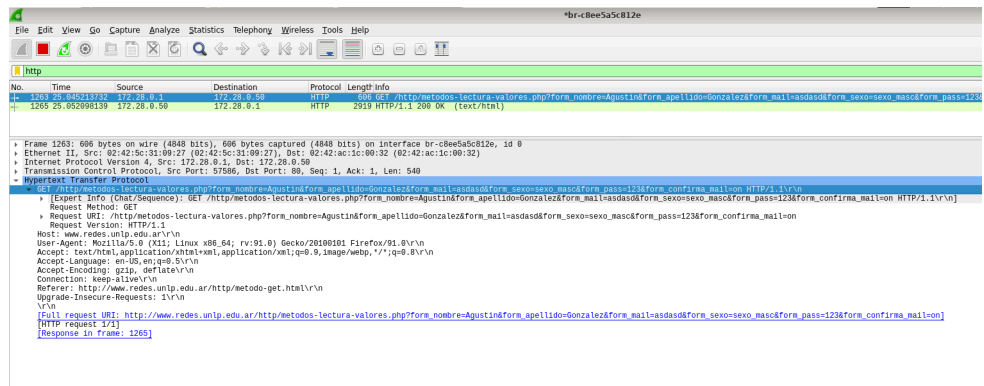
```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
  <div id="wrap">
    <div class="navbar navbar-inverse navbar-fixed-top">
      <div class="navbar-inner">
      </div>
    </div>
    <div class="container">
      <h1>Métodos HTTP: GET</h1>
      <p>
        <form method="GET" action="metodos-lectura-valores.php">
        </form>
      </p>
    </div>
  </div>
  <div id="footer">
    <div class="container">
      <p>Redes y Comunicaciones</p>
    </div>
  </div>
</body>
</html>
```

-POST:

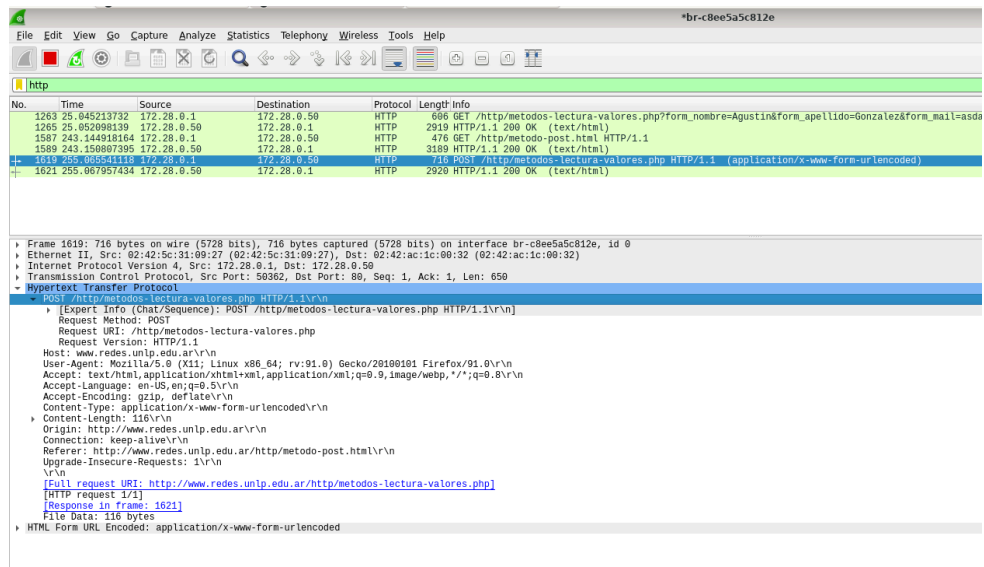
```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
  <div id="wrap">
    <div class="navbar navbar-inverse navbar-fixed-top">
      <div class="navbar-inner">
      </div>
    </div>
    <div class="container">
      <h1>Métodos HTTP: POST</h1>
      <p>
        <form method="POST" action="metodos-lectura-valores.php">
          <table width="80" border="0">
          </table>
        </form>
      </p>
    </div>
  </div>
  <div id="footer">
    <div class="container">
      <p>Redes y Comunicaciones</p>
    </div>
  </div>
</body>
</html>
```

- Utilizando el analizador de paquetes Wireshark capture los paquetes enviados y recibidos al presionar el botón Enviar.

-GET:



-POST:



No.	Time	Source	Destination	Protocol	Length	Info
1263	25.045213732	172.28.0.1	172.28.0.50	HTTP	696	GET /http/metodos-lectura-valores.php?form_nombre=Agustin&form_apellido=Gonzalez&form_mail=as
1265	25.952098139	172.28.0.50	172.28.0.1	HTTP	2919	HTTP/1.1 200 OK (text/html)
1587	243.144918164	172.28.0.1	172.28.0.50	HTTP	476	GET /http/metodo-post.html HTTP/1.1
1589	243.150807395	172.28.0.50	172.28.0.1	HTTP	3189	HTTP/1.1 200 OK (text/html)
1619	255.065541119	172.28.0.1	172.28.0.50	HTTP	716	POST /http/metodos-lectura-valores.php HTTP/1.1 (application/x-www-form-urlencoded)
1621	255.067957434	172.28.0.50	172.28.0.1	HTTP	2920	HTTP/1.1 200 OK (text/html)

Frame 1589: 3189 bytes on wire (25512 bits), 3189 bytes captured (25512 bits) on interface br-c8ee5a5c812e, id 0	
Ethernet II, Src: 02:42:ac:1c:00:32 (02:42:ac:1c:00:32), Dst: 02:42:5c:31:09:27 (02:42:5c:31:09:27)	
Internet Protocol Version 4, Src: 172.28.0.50, Dst: 172.28.0.1	
Transmission Control Protocol, Src Port: 80, Dst Port: 38548, Seq: 1, Ack: 411, Len: 3123	
Hypertext Transfer Protocol	
[Expert Info (Chat/Sequence): HTTP/1.1 200 OK\r\n] Response Version: HTTP/1.1 Status Code: 200 [Status Code Description: OK] Response Phrase: OK Date: Thu, 03 Sep 2020 03:18:41 GMT\r\n Server: Apache/2.4.56 (Unix)\r\n Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT\r\n ETag: "b217-5f7457d6d4f60"\r\n Accept-Ranges: bytes\r\n Content-Length: 2839\r\n Keep-Alive: timeout=5, max=100\r\n Connection: Keep-Alive\r\n Content-Type: text/html\r\n [HTTP response 1/1] [Time since request: 0.005889231 seconds] Request in frame: 1587 Request URI: http://www.redes.unlp.edu.ar/http/metodo-post.html File Data: 2839 bytes Line-based text data: text/html (78 lines)	

4. ¿Qué diferencias detectó en los mensajes enviados por el cliente?

-La diferencia es que si se trata de GET, la informacion se envia como parametro de la URL, mientras que en POST se envia en el body de la solicitud (separado)

5. ¿Observó alguna diferencia en el browser si se utiliza un mensaje u otro?

-En GET puedes ver en la URL los parametros enviados y en POST no, se mantiene la misma URL que se tiene antes de apretar el boton

18. Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP.

-Las cookies permiten a los sitios seguir la pista a los usuarios. Utiliza 4 componentes →

- 1) linea de cabecera de la cookie en el mensaje de respuesta HTTP
- 2) linea de cabecera de la cookie en el mensaje de solicitud HTTP
- 3) archivbo de cookies almacenado en el sistema terminal del usuario y gestionado por el navegador
- 4) base de datos back-end en el sitio web

-El principal uso de las cabeceras "Set-Cookie" y "Cookie" en HTTP es permitir a los servidores/clientes mantener un estado persistente entre

solicitudes y respuestas. La cookie se introduce al cliente mediante el mensaje en el header "Set-Cookie:" indicando el par (nombre,valor). El cliente en cada requerimiento luego de haber almacenado la Cookie se la envia al servidor con el header "Cookie:"

**19. ¿Cuál es la diferencia entre un protocolo binario y uno basado en texto?
¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?**

-Protocolo binario → Transmite datos en patrones de bits que las maquinas entienden directamente. Mas eficiente en terminos de velocidad/uso de ancho de banda ya que no desperdicia espacio en caracteres o interpretacion de texto legible por humanos. Es utilizado mas por HTTP/2

-Protocolo basado en texto → Transmite datos como cadenas legibles usando caracteres como letras,numeros, simbolos. Esta normalmente coficiado en ASCII o UTF-8. Por mas que la comunicacion sea mas comprensible para nosotros, es menos eficiente a nivel de maquina en terminos de velocidad y uso de ancho de banda, ya que justamente se requiere interpretar la informacion y traducirla a datos en formato binario. Es utilizado por HTTP/1.0 y HTTP/1.1

20. Responder las siguientes preguntas:

a. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0? ¿Qué sucede en HTTP/2? (Ayuda:

<https://undertow.io/blog/2015/04/27/An-in-depth-overview-of-HTTP2.html> para HTTP/2)

-La cabecera "Host" en HTTP/1.1 cumple la funcion de indicar el nombre del domino al que se le esta haciendo la request dentro del encabezado de la peticion y es obligatoria. Es importante en servidores que hostean muchos sitios web en una misma direccion IP, ya que permite al servidor saber a que sitia se esta dirigiendo la solicitud.

-En HTTP/1.0 esta cabecera no es obligatoria pero puede ser necesaria dependiendo de la configuracion del servidor web. Al no ser obligatoria, si la cabecera Host no esata presente en HTTP/1.0 el servidor va a asumir que la solicitud esta destinada al dominio asociado con la IP del servidor

-HTTP/2 reemplaza "Host" por ":authority"

b. En HTTP/1.1, ¿es correcto el siguiente requerimiento?

GET /index.php HTTP/1.1

User-Agent: curl/7.54.0

-No es correcto ya que falta la cabecera "Host" que es obligatoria en HTTP/1.1, por ejemplo:

```
GET /index.php HTTP/1.1
Host: www.info.unlp.edu.ar
User-Agent: curl/7.54.0
```

c. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https?

GET /index.php HTTP/1.1

Host: www.info.unlp.edu.ar

-Esa solicitud en HTTP/2 quedaria:

```
:method: GET
:scheme: https
:authority: www.info.unlp.edu.ar
:path: /index.php
```

(Creo que no hace falta este orden: method-> scheme-> authority-> path)

Ejercicio integrador

```
curl -X ?? www.redes.unlp.edu.ar/??
> HEAD /metodos/ HTTP/1.0
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.54.0

< HTTP/1.1 200 OK
< Server: nginx/1.4.6 (Ubuntu)
< Date: Wed, 31 Jan 2018 22:22:22 GMT
< Last-Modified: Sat, 20 Jan 2018 13:02:41 GMT
< Content-Type: text/html; charset=UTF-8
< Connection: close
```

1. ¿Qué método está utilizando? Dicho método, ¿retorna el recurso completo solicitado?
2. ¿Cuál es el recurso solicitado?
3. ¿El método funcionó correctamente?

4. Si la solicitud hubiera llevado un encabezado que diga:
If-Modified-Since: Sat, 20 Jan 2018 13:02:41 GMT
¿Cuál habría sido la respuesta del servidor web? ¿Qué habría hecho el navegador en este caso?